Dynamic Allocation

Dynamic Allocation

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Dynamic Allocation

```
int *p = (int*) malloc(sizeof(int)); *p = 100; int *q = p; int a = *p; *q = 110;
```

What are values of a, *p, *q assuming malloc succeeds?

a) 110 110 110

b) 100 110 110

c) 100 100 110

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Dynamic Allocation

```
int *p = (int*) malloc(sizeof(int)); *p = 100; int *q = p; int a = *p; *q = 110;
```

What are values of a, *p, *q assuming malloc succeeds?

a) 110 110 110

b) 100 110 110

c) 100 100 110

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Dynamic Allocation

```
int *p = (int*) malloc(sizeof(int)); *p = 100; int *q = p;

   free(q); printf("%d ", *p); // assume malloc succeeds?
```

a) 100

b) Cannot call free on q as there was allocation on p

c) Undefined behaviour as p is a dangling pointer

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Dynamic Allocation

```
int *p = (int*) malloc(sizeof(int)); *p = 100; int *q = p;

   free(q); printf("%d ", *p); // assume malloc succeeds?
```

a) 100

b) Cannot call free on q as there was allocation on p

c) Undefined behaviour as p is a dangling pointer

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Dynamic Allocation

```
void foo() {
    int *p = (int*)malloc(sizeof(int)); // assume malloc succeeds
    // some operations here

}
```

a) Clean code as p is removed from the stack at the end of the block

b) Results in Dangling pointer at the end of the block

c) Results in garbage; garbage collector makes it useful again

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Dynamic Allocation

```
void foo() {
    int *p = (int*)malloc(sizeof(int)); // assume malloc succeeds
    // some operations here

}
```

a) Clean code as p is removed from the stack at the end of the block

b) Results in Dangling pointer at the end of the block

c) Results in garbage; garbage collector makes it useful again

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Dynamic Allocation

```
struct node { int key; struct node next; } x;
```

a) Will not compile; a structure cannot have a structure of the same type within it

b) No error; struct will have the combined size of an int and a pointer

c) No error; x.next will be a structure with key and next as the fields

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Dynamic Allocation

```
struct node { int key; struct node next; } x;
```

a) Will not compile; a structure cannot have a structure of the same type within it

b) No error; struct will have the combined size of an int and a pointer

c) No error; x.next will be a structure with key and next as the fields

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Dynamic Allocation

```
struct node { int key; struct node* next; } x;

x.key = 10; x.next = &x;
```

a) Will not compile; x.next and &x are not of the same type

b) x.next.key is 10

c) x.next->next will be a dangling pointer

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Dynamic Allocation

```
struct node { int key; struct node* next; } x;

x.key = 10; x.next = &x;
```

a) Will not compile; x.next and &x are not of the same type

b) x.next.key is 10

c) x.next->next will be a dangling pointer

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Dynamic Allocation

- C does not support any operator for memory allocation or deallocation
- These are realized by using the standard library functions
- void* malloc(int size);
    - Is a low level function
    - Should indicate how many bytes are required
    - Allocates a block of so many bytes and returns a malleable pointer to it
    - Does book keeping – remembers the size of the block allocated
    - You may cast it to any type you want
    - Returns NULL when it fails

© Confidential Information of PES University. All Rights Reserved - 2026

Dynamic Allocation

- void free(void *ptr);
  - Frees the block pointed to by ptr
  - Makes the block of memory useful again
  - Observe that we do not specify the size of the block allocated
  - Uses book keeping information created while allocating
  - Calling free on a pointer value not returned by malloc or calling second time are undefined behaviour
  - Free(NULL) is a no op.

© Confidential Information of PES University. All Rights Reserved - 2026

Dynamic Allocation

- Garbage : location with no access
- Happens only with dynamic allocation
- Happens when memory is allocated and not freed
- Happens when the pointer holding the pointer value to the allocated block is changed or dies
- Garbage becomes memory leak in 'C' as 'C' does not support garbage collector
- All the garbage is collected when the program terminates

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)